№7_Архитектура_вычислительных_систем

План

- Команды условного перехода
  - Таблица команд
  - Флаги
    - CF
    - OF
    - ZF
    - SF
    - PF
    - AF
    - DF
    - TF
  - Задача
  - Loop
  - Loopne/loopz

Команды условного перехода

```
cpm <операнд1>, <операнд2>
```

Команда `cmp` *выполняет вычитание так же как* и команда `sub`, *но* она *только устанавливает флаги и не изменяет значение операндов*.

Таблица команд

Команды условного перехода - это *команды, выполняющие переход на метку в зависимости от типа команды и* значения

флагов.

Таблица команд условного перехода:

Тип операндов	Команда	Условие	Значения флагов
Любые	`JE`	`a = b`	`ZF = 1`
Любые	`JNE`	`a ≠ b`	`ZF = 0`
Знаковые	`JL/JNGE`	`a < b`	`SF ≠ OF`
Знаковые	`JLE/JNG`	`a ⩽ b`	`SF ≠ OF` или `ZF = 1`
Знаковые	`JG/JNLE`	`a > b`	`SF ≠ OF` и `ZF = 0`
Знаковые	`JGE/JNL`	`a ⩾ b`	`ZF = OF`
Беззнаковые	`JB/JNAE`	`a < b`	`CF = 1`
Беззнаковые	`JBE/JNA`	`a ⩽ b`	`CF = 1` или `ZF = 1`
Беззнаковые	`JA/JNBE`	`a > b`	`CF = 0` и `ZF = 0`
Беззнаковые	`JAE/JNB`	`a ⩾ b`	`CF = 0`

Пример:

```
cmp a, b
je j_lable1           ; a=b
dec esi

j_lable1:
     inc ebx
```

Флаги

CF

Флаг переноса `CF` *устанавливается при заёме или переносе старшего* знакового *бита*.

Переполнение регистра:

```
mov al, 11111111b
add al, 2                ; CF == 1, al == 0000 0001
```

Истощение регистра:

```
mov al, 00000000b
sub al, 5                ; CF == 1
```

OF

Флаг переполнения `OF` *устанавливается при заёме или переносе в старший* знаковый *бит*. По сути *при смене знака*.

```
mov bl, 01111111b        ;  127 (знаковое)
add bl, 1                ; -128, OF == 1
```

ZF

Флаг нуля `ZF` *устанавливается когда результат выполнения последней операции равен нулю*.

```
mov ecx, 3
sub ecx, 3               ; ZF == 1
```

SF

Флаг знака `SF` *дублирует значение старшего* знакового *бита*.

```
xor eax, eax
add al, 1                ; SF == 0
sub al, 2                ; SF == 1
```

PF

Флаг чётности `PF` *устанавливается если младший байт результата содержит чётное число единиц*.

```
xor eax, eax
add ax, 111111b          ; PF = 1
sub ax, 1                ; PF = 0
```

AF

Флаг `AF` *устанавливается при заёме или переносе из третьего бита в четвёртый*.

```
mov ax, 1111b            ;
add ax, 1                ; AF == 1
```

DF

Флаг направления `DF` *устанавливается вручную*. Он используется для работы некоторых команд.

TF

Флаг трассировки `TF` *разрешает отладку программы*.

Задача

Напишем подпрограмму для вычисления максимума трёх чисел.

Сначала на `c`:

```c
int max(int a, int b, int c) {
     inr r;

     if(a > b) r = a;
     else r = b;

     if(r < c) r = c;

     return r;
}
```

`asm`:

```
.data
a dd 10
b dd 5
d dd -10

.code
push d
push b
push a

call max

max prog
     mov eax, [esp + 4]        ; r = a
     cmp eax, [esp + 8]        ; r - b
     jg j_if                   ; a > b
     mov eax, [esp + 8]        ; r = b

i_if:
     cmp eax, [esp + 12]       ; r - c
     jg j_exit
     mov eax, [esp + 12]       ; r = c

j_exit:
     RET 12

max endp
```

Loop

```
loop <метка>
```

Команда `loop` сначала уменьшает содержимое `ecx` на единицу, затем она сравнивает если `ecx` больше нуля, то управление передаётся на метку. Если `ecx` равно нулю, то управление передаётся следующей за `loop` команде.

Loopne/loopz

Если `ecx` больше нуля и `ZF` равен единице, то управление передаётся на метку. Иначе выполняется следующая команда.

Выход из цикла происходит если `ecx` равно нулю или `ZF` обнулился.

По сути такая реализация позволяет досрочно завершить цикл с помощью флага.